

STRIPE PROJECT

1. OLTP Data System

D8.1. OLTP SQL Queries Examples

Nicolas Pichon - AIA RNCP 38777 / BC02 / D8.1 : OLTP Queries / v28 - 2026/10/13.

D8.1.1. Contexte

Exemples de requêtes sur les données transactionnelles dans les domaines suivants :

- analyse de revenu,
- détection de fraude,
- segmentation client.

Le modèle physique de référence est défini dans l'annexe B [D2-B].

Le moteur cible est *PostgreSQL*.

Note concernant les index sollicités

Les requêtes suivantes s'appuient exclusivement sur des indexes déjà présents dans le modèle physique ; aucun index supplémentaire n'est requis :

- (merchant_id, created_at) sur Transaction couvre D8.2.1 et D8.4.1 ;
- risk_level_code sur FraudScore couvre D8.3.1 & D8.3.2.
- D8.2.2 et D8.4.2 n'ont pas de filtre temporel ou marchand assez sélectif pour tirer parti d'un index composé (un balayage complet sans index reste acceptable à leur volume).

D8.1.2. Analyse de revenu

D8.1.2.1. Revenus et commissions récents des marchands

Récupérer le montant total encaissé, la commission *Stripe* prélevée, et le revenu net du marchand, par marchand, sur les 30 derniers jours, sur les transactions réussies uniquement.

```
SELECT
    m.merchant_id,
    m.legal_name AS merchant_name,
    t.currency_code AS currency,
    COUNT(t.transaction_id) AS transactions,
    SUM(t.amount) AS total_amount,
    SUM(t.fee_amount) AS fee_amount,
    SUM(t.amount - t.fee_amount) AS merchant_amount
FROM Transaction t
JOIN Merchant m ON m.merchant_id = t.merchant_id
JOIN TransactionStatus ts ON ts.status_code = t.status_code
WHERE ts.status_code = 'successful'
    AND t.created_at >= now() - INTERVAL '30 days'
GROUP BY m.merchant_id, m.legal_name, t.currency_code
ORDER BY total_amount DESC;
```

D8.1.2.2. Revenus des produits avec ventilation ponctuel/récurrent

Distinguer les transactions (réussies) issues d'un abonnement des achats ponctuels (cf. suivi de performance des produits, exigé dans le business case).

```
SELECT
    p.product_id,
```

```

p.name AS product_name,
CASE WHEN t.subscription_id IS NULL THEN 'ponctuel' ELSE 'recurrent' END
    AS revenue_type,
t.currency_code AS currency,
COUNT(*) AS revenues,
SUM(t.amount) AS total_amount
FROM Transaction t
JOIN Product p ON p.product_id = t.product_id
JOIN TransactionStatus ts ON ts.status_code = t.status_code
WHERE ts.status_code = 'successful'
GROUP BY p.product_id, p.name, revenue_type, t.currency_code
ORDER BY p.product_id, revenue_type;

```

D8.1.3. Détection de fraude

D8.1.3.1. Transactions à risque élevé nécessitant une revue manuelle

Exploiter directement la logique portée par `FraudRiskLevel` au lieu d'utiliser un seuil arbitraire.

```

SELECT
    t.transaction_id,
    t.merchant_id,
    t.amount,
    t.currency_code,
    fs.anomaly_score,
    frl.label AS risk_level,
    t.device_type,
    t.ip_geolocation,
    t.created_at
FROM Transaction t
JOIN FraudScore fs ON fs.transaction_id = t.transaction_id
JOIN FraudRiskLevel frl ON frl.risk_level_code = fs.risk_level_code
WHERE frl.requires_manual_review = true
ORDER BY fs.anomaly_score DESC;

```

D8.1.3.2. Validation a posteriori du modèle de scoring

Comparer les scores de fraude aux rétro-facturations réellement survenues (ex: un score de fraude `low` suivi d'une rétro-facturation `fraud` signale un faux négatif du modèle).

Seule requête du domaine qui évalue la fiabilité du système plutôt que de l'exploiter directement.

```

SELECT
    frl.label AS predicted_risk_level,
    COUNT(*) AS transactions,
    COUNT(cb.chargeback_id) AS chargebacks,
    COUNT(cb.chargeback_id) FILTER (WHERE crc.category = 'fraud') AS frauds,
    ROUND( 100.0 * COUNT(cb.chargeback_id) FILTER (WHERE crc.category = 'fraud') / NULLIF(COUNT(*), 0),
3) AS negative_false_rate_in_percent
FROM Transaction t
JOIN FraudScore fs ON fs.transaction_id = t.transaction_id
JOIN FraudRiskLevel frl ON frl.risk_level_code = fs.risk_level_code
LEFT JOIN Chargeback cb ON cb.transaction_id = t.transaction_id
LEFT JOIN ChargebackReasonCode crc ON crc.reason_code = cb.reason_code
GROUP BY frl.label
ORDER BY negative_false_rate_in_percent DESC NULLS LAST;

```

D8.1.4. Segmentation client

D8.1.4.1. Segmentation *RFM*

Classer les clients d'un marchand *donné* (i.e. défini comme paramètre de la requête) en quartiles sur les axes *RFM* (*Recency*, *Frequency*, *Monetary value*).

```

WITH customer_stats AS (
    SELECT
        c.customer_id,
        c.email,
        t.currency_code AS currency,
        MAX(t.created_at) AS last_transaction,
        COUNT(*) AS transactions,
        SUM(t.amount) AS total_amount
    FROM Customer c
    JOIN Transaction t ON t.customer_id = c.customer_id
    WHERE t.merchant_id = :merchant_id
        AND t.status_code = 'successful'
    GROUP BY c.customer_id, t.currency_code
)
SELECT
    customer_id,
    email,
    transactions,
    total_amount,
    NTILE(4) OVER (ORDER BY last_transaction DESC) AS quartile_recency,
    NTILE(4) OVER (ORDER BY transactions DESC) AS quartile_frequency,
    NTILE(4) OVER (ORDER BY total_amount DESC) AS quartile_amount
FROM customer_stats
ORDER BY quartile_amount, quartile_frequency, quartile_recency;

```

D8.1.4.2. Segmentation par valeur et par méthode de paiement

Combiner la valeur totale du client et son moyen de paiement principal (par défaut).

Utile pour cibler des campagnes par type de moyen de paiement.

Seule requête du domaine à croiser deux dimensions de segmentation au lieu d'une seule.

```

WITH customer_segment AS (
    SELECT
        c.customer_id,
        t.currency_code AS currency,
        SUM(t.amount) AS total_amount,
        CASE
            WHEN SUM(t.amount) >= 1000 THEN 'top'
            WHEN SUM(t.amount) >= 200 THEN 'regular'
            ELSE 'occasional'
        END AS segment
    FROM Customer c
    JOIN Transaction t ON t.customer_id = c.customer_id
    AND t.status_code = 'successful'
    GROUP BY c.customer_id, t.currency_code
)
SELECT
    customer_segment.segment,
    customer_segment.currency,
    pmt.label AS default_payment_method_type,
    COUNT(*) AS customers,
    ROUND(AVG(customer_segment.total_amount), 2) AS avg_customer_ticket
FROM customer_segment
JOIN PaymentMethod pm ON pm.customer_id = customer_segment.customer_id
    AND pm.is_default = true
JOIN PaymentMethodType pmt ON pmt.type_code = pm.type_code
GROUP BY customer_segment.segment, customer_segment.currency, pmt.label
ORDER BY customer_segment.segment, customers DESC;

```

Point d'attention : les seuils de segmentation (≥ 1000 , ≥ 200) restent des valeurs absolues, indépendantes de la devise — 1000 USD et 1000 JPY ne représentent pas la même valeur réelle. Rendre `currency` visible dans le résultat expose ce problème plutôt que de le masquer, mais ne le résout pas : une vraie segmentation multi-devises demanderait soit des seuils

par devise, soit une conversion vers une devise de référence (déjà résolue côté OLAP via `DimReferenceExchangeRate` ,
absente ici puisque D8.1 opère directement sur l'OLTP non converti).

Références

Ressources

- [R0] [Stripe Business Case](#)
-

Annexes

- [D2-B] [OLTP Physical Model](#)